Software Quality Assurance

Lecture # 6

Today's Lecture

- We'll discuss different topics related to software quality
- This is the last lecture in the first phase of this course

Quality Measurements

Quality Measurement Questions

- What should be measured for quality?
- How often should quality measurement be taken and reported?

Quality Measurement Categories

- Measurement of defects or bugs in software
  - 100% of software projects
- Measurement of user-satisfaction levels
  - Only for software projects where clients can be queried and actually use the software consciously

Software Defect Quality Measurements - 1

- Defect volumes (by product, by time period, by geographic region)
- Defect severity levels
- Special categories (invalid defects, duplicates, un-duplicatable problems)
- Defect origins (i.e., requirements, design, code, documents, or bad fixes)

Software Defect Quality Measurements - 2

- Defect discovery points (i.e., inspections, tests, customer reports, etc.)
- Defect removal efficiency levels
- Normalized data (i.e., defects per function point or per KLOC)
- Causative factors (i.e., complexity, creeping requirements, etc.)

Software Defect Quality Measurements - 3

- Defect repair speeds or intervals from the first report to the release of the fix

Software User-Satisfaction Quality Measurements - 1

- User perception of quality and reliability
- User perception of features in the software product
- User perception of ease of learning
- User perception of ease of use
- User perception of customer support
- User perception of speed of defect repairs

Software User-Satisfaction Quality Measurements - 2

- User perception of speed of adding new features
- User perception of virtues of competitive products
- User perception of the value versus the cost of the package

Who Measures User-Satisfaction?

- Marketing or sales organization of the software company
- User associations
- Software magazines
- Direct competitors
- User groups on the internet, etc.
- Third-party survey groups

Gathering User-Satisfaction Data

- Focus groups of customers
- Formal usability laboratories
- External beta tests
- Requests from user associations for improvements in usability
- Imitation of usability features of competitive or similar products by other vendors

Barriers to Software Quality Measurement

- Lack of understanding of need to measure quality
- Often technical staff shies away from getting their work measured
- Historically, “lines of code” or LOC and “cost per defect” metrics have been used, which are a poor way of measuring software quality

Object-Oriented Quality Levels

- OO technology is being adopted world-wide with a claim that it produces better quality software products
- OO technology has a steep learning curve, and as a result it may be difficult to achiever high quality software
- More data needs to be reported
- UML may play a significant role

Orthogonal Defect Reporting - 1

- The traditional way of dealing with software defects is to simply aggregate the total volumes of bug reports, sometimes augmented by severity levels and assertions as to whether defects originated in requirements, design, code, user documents, or as “bad fixes”

Orthogonal Defect Reporting - 2

- In the orthogonal defect classification (ODC) method, defects are identified using the following criteria
  - Detection method (design/code inspection, or any of a variety of testing steps)
  - Symptom (system completely down, performance downgraded, or questionable data integrity)
  - Type (interface problems, algorithm errors, missing function, or documentation error)
  - Trigger (start-up / heavy utilization / termination of application, or installation)
  - Source (version of of the projects using normal configuration control identification)

Outsourcing and Software Quality

- Outsourcing in software industry is done in a variety of ways
- Every situation introduces new challenges for development of high quality software
- Software quality metrics must be mentioned in the outsourcing contract

Quality Estimating Tools - 1

- Estimating defect potentials for bugs in five categories (requirements, design, coding, documentation, and bad fixes)
- Estimating defect severity levels into four categories, ranging from 1 (total or catastrophic failure) to severity 4 (minor or cosmetic problem)

Quality Estimating Tools - 2

- Estimating the defect removal efficiency levels of various kinds of design reviews, inspections, and a dozen kinds of testing against each kind and severity of defects
- Estimating the number and severity of latent defects present in a software application when it is delivered to users

Quality Estimating Tools - 3

- Estimating the number of user-reported defects on an annual basis for up to 20 years
- Estimating the reliability of software at various intervals using mean-time to failure (MTTF) and/or mean-time between failures (MTBF) metrics

Quality Estimating Tools - 4

- Estimating the “stabilization period” or number of calendar months of production before users can execute the application without encountering severe errors.
- Estimating the efforts and costs devoted to various kinds of quality and defect removal efforts such as inspections, test-case preparation, defect removal, etc.

Quality Estimating Tools - 5

- Estimating the number of test cases and test runs for all testing stages
- Estimating maintenance costs for up to 20 years for fixing bugs (also for additions)
- Estimating special kinds of defect reports including duplicates and invalid reports which trigger investigative costs but no repair costs

Quality Process Metrics

- Defect arrival rate
- Test effectiveness
- Defects by phase
- Defect removal effectiveness
- Defect backlog
- Backlog management index
- Fix response time
- Percent delinquent fixes
- Defective fixes

Product Metrics

- Defect density
- Defects by severity
- Mean time between failures
- Customer-reported problems
- Customer satisfaction

Function Point Metric - 1

- It was developed at IBM and reported to public in 1979
- It is a way of determining the size of a software application by enumerating and adjusting five visible aspects that are of significance to both users and developers

Function Point Metric - 2

- Inputs that enter the application (i.e., Input screens, forms, commands, etc.)
- Outputs that leave the application (i.e., Output screens, reports, etc.)
- Inquiries that can be made to the application (i.e., Queries for information)
- Logical files maintained by the application (i.e., Tables, text files, etc.)

Function Point Metric - 3

- Interfaces between the application and others (i.e., shared data, messages, etc.)

Function Point Metric - 4

- Once the raw total of these five factors has been enumerated, then an additional set of 14 influential factors are evaluated for impact using a scale that runs from 0 (no impact) to 5 (major impact)

Litigation and Quality

- Relevant factors for software quality
  - Correctness, reliability, integrity, usability, maintainability, testability, understandability
- Irrelevant factors for software quality
  - Efficiency, flexibility, portability, reusability, interoperability, security
- It is important to narrow down the scope of quality definition similar to hardware warranties

Schedule Pressure and Quality

- Healthy pressure
  - Motivates and keeps morale of the personnel high
- Excessive pressure
  - Has serious negative impact on the morale of personnel
  - Can lead to low quality software

- Project life cycle quality assurance activities are process oriented, in other words, linked to completion of a project phase, accomplishment of a project milestone, and so forth
- The quality assurance activities will be integrated into the development plan that implements one ore more software development models – waterfall, prototyping, spiral, etc.

- The SQA planners for a project are required to determine
  - The list of quality assurance activities needed for a project
  - For each quality assurance activity
    - Timing
    - Who performs the activity and the resources needed
    - Resources required for removal of defects and introduction of changes

A Word of Caution

- Some development plans, QA activities are spread throughout the process, but without any time allocated for their performance or for the subsequent removal of defects. As nothing is achieved without time, the almost guaranteed result is delay, caused by “unexpectedly” long duration of the QA process
- Hence, the time allocated for QA activities and the defects corrections work that follow should be examined

- The intensity of the quality assurance activities planned, indicated by the number of required activities, is affected by project and team factors

Project Factors

- Magnitude of the project
- Technical complexity and difficulty
- Extent of reusable software components
- Severity of failure outcomes if the project fails

Team Factors

- Professional qualification of the team members
- Team acquaintance with the project and its experience in the area
- Availability of staff members who can professionally support the team
- Familiarity with team members, in other words the percentage of new staff members in the team

Why Error-Prone Modules?

- Excessive schedule pressure on the programmers
- Poor training or lack of experience in structured methods
- Rapidly creeping requirements which trigger late changes
- High complexity levels with cyclomatic ranges greater than 15

“Good Enough” Software Quality - 1

- Rather than striving for zero-defect levels or striving to exceed in 99% in defect removal efficiency, it is better to ship software with some defects still present in order to speed up or shorten time to market intervals
- Developed by the fact that major commercial software companies have latent software bugs in their released products

“Good Enough” Software Quality - 2

- Major commercial software companies have cumulative defect removal efficiency of 95% (and 99% on their best projects)
- This concept is very hazardous for ordinary companies, which usually have their defect removal efficiency level between 80%-85%
- Quality will be decrease for these companies

Data Quality - 1

- Extremely important to understand issues of data quality
- Data results in (useful | useless) information
- Usually, governments are holders of largest data banks (are they consistent?)
- Companies are increasingly using data to their advantage over competitors

Data Quality - 2

- Data warehouses present a unique challenge to keep data consistent
- Another problem is the interpretation of data

References

- Software Quality: Analysis and Guidelines for Success by Capers Jones
- Customer-Oriented Software Quality Assurance by Frank Ginac
- A Practitioner’s Approach to Software Engineering by Roger Pressman